Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
Федеральное государственное бюджетное общеобразовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Расчетно-графическая работа
по дисциплине: «Базы данных»

Выполнили: студенты гр.ИКС-432
Шаламов Н. и Пастухов А.
Проверил: старший преподаватель
кафедры ТС и ВС
Брагин К.И

Новосибирск, 2026

Содержание

1. Цель работы 3
2. Выполнение работы 4
2.0 Подготовка и описание предметной области 4
2.2 Создание схемы БД (schema.sql) 6
2.2.1 Создание таблиц 6
2.2.2 Добавление внешних ключей и ограничение целостности 8
2.2.3 Создание индексов 10
2.2 Наполнение тестовыми данными (data.sql) 12
2.2.2 Статистика наполнения 12
2.2.2 Примеры добавленных данных 12
2.3 Визуализация структуры БД (diagram) 15
2.4 Аналитические запросы (queries.sql) 20
2.4.1 Q-01 .. Q-10 - базовый и средний уровень 20
2.4.2 Q-11 .. Q-20 - расширенный аналитический стек 23
2.5 Запуск через Docker Compose 26
2.5.1 Файл конфигурации 26
2.5.2 Порядок запуска 26
2.5.3 Автоматическая инициализация 27
2.6 Результат выполнения 28
3. Выводы 29

1. Цель работы

1.1 Спроектировать авторскую реляционную базу данных по тематике управления IT-инфраструктурой дата-центра, включающую сущности серверов, стоек, сервисов, инцидентов и сетевой инфраструктуры.
1.2 Реализовать разработанную структуру базы данных в СУБД PostgreSQL с использованием SQL-скриптов создания схемы, ограничений целостности и тестовых данных.
1.3 Визуализировать логическую структуру базы данных в виде ER-диаграммы с отображением сущностей и связей между ними.
1.4 Разработать набор SQL-запросов, демонстрирующих функциональные возможности, аналитические операции и обработку данных в созданной системе.
1.5 Освоить и применить расширенные возможности PostgreSQL, включая хранимые функции, триггеры аудита и контейнеризацию среды выполнения с использованием Docker Compose.

2. Выполнение работы

2.1 Подготовка и описание предметной области

Спроектирована база данных для централизованного учёта и мониторинга IT-инфраструктуры дата-центра. Предметная область включает следующие основные сущности и бизнес-процессы:

- **Дата-центры (datacenters)** - физические площадки с указанием местоположения, уровня надёжности по классификации Tier (1-4) и контактной информации. В системе зарегистрированы три площадки: DC-Alpha (Москва), DC-Beta (Санкт-Петербург), DC-Gamma (Новосибирск).
- **Стойки (racks)** - физическое размещение оборудования внутри дата-центра. Каждая стойка привязана к площадке, имеет уникальную метку, номер ряда, максимальную высоту в юнитах и ограничение по мощности.
- **Сотрудники (employees)** - инженеры, администраторы, операторы NOC, менеджеры и сотрудники безопасности. Хранятся контактные данные и роль каждого сотрудника.
- **Вендоры (vendors)** - отечественные поставщики оборудования и ПО: YADRO, Элтекс, Булат, Т-Ком. Хранятся контакты службы поддержки.
- **Контракты SLA (contracts)** - договоры технической поддержки с вендорами, содержащие номер договора, сроки действия и гарантированное время реакции (SLA в часах).
- **Серверы (servers)** - основная сущность: физические серверы с характеристиками (CPU, RAM, хранилище, потребляемая мощность), привязкой к стойке, договору поддержки и ответственному инженеру.
- **Компоненты серверов (server_components)** - детальный учёт внутреннего оборудования: процессоры, модули памяти, диски, сетевые карты, блоки питания. Каждый компонент имеет статус (healthy/failed/replaced).
- **IPAM** - учёт IP-адресов (ip_addresses) с поддержкой типов INET/CIDR PostgreSQL. Хранятся тип адреса (management, production, public), принадлежность к подсети и признак виртуального IP (VIP).
- **Сервисы (services)** - логические сервисы, работающие на серверах (Nginx, PostgreSQL, Redis, OpenVPN, DNS и др.), с указанием порта и текущего статуса.
- **Инциденты (incidents)** - журнал аварий и сбоев с уровнем критичности (low/medium/high/critical), статусом, временем создания и разрешения.
- **Рабочие заметки к инцидентам (incident_updates)** - лента обновлений (worklog) по инциденту для Post-Mortem анализа.
- **Связь инцидентов и сервисов (incident_services)** - таблица M:N, фиксирующая, какие сервисы были затронуты каждым инцидентом.

- **Климатическая телеметрия стоек (rack_climate_metrics)** - IoT-метрики: энергопотребление и температура стоек в реальном времени.
- **Журнал обслуживания (maintenance_logs)** - регистрация плановых и внеплановых работ: замена дисков, обновление прошивки, апгрейд памяти, диагностика.

Бизнес-правила, заложенные в схему:

- Сервер не может существовать без стойки.
- При удалении сервера все его сервисы и компоненты удаляются автоматически (ON DELETE CASCADE).
- Инцидент обязательно привязан к серверу; время разрешения не может быть раньше времени создания (CHECK constraint).
- В одну юнит-позицию стойки нельзя установить два сервера (UNIQUE по rack_id + rack_unit_position).
- При удалении вендора его контракты сохраняются (ON DELETE RESTRICT), а у серверов обнуляется ссылка на договор (ON DELETE SET NULL).

2.2 Создание схемы БД (schema.sql)

Разработан единый скрипт schema.sql, который создаёт все 14 таблиц, ограничения целостности и индексы. Скрипт начинается с каскадного удаления существующих таблиц для обеспечения консистентности при повторном выполнении:

```
DROP TABLE IF EXISTS incident_updates CASCADE;
DROP TABLE IF EXISTS incident_services CASCADE;
DROP TABLE IF EXISTS datacenters CASCADE;
DROP TABLE IF EXISTS employees CASCADE;
```

2.2.1 Создание таблиц

Таблица сотрудников:

```
CREATE TABLE employees (
    id          SERIAL PRIMARY KEY,
    full_name   VARCHAR(150) NOT NULL,
    email       VARCHAR(150) NOT NULL UNIQUE CHECK (email LIKE '%@%.%'),
    phone       VARCHAR(20)  NOT NULL UNIQUE,
    role        VARCHAR(60)  NOT NULL CHECK (
                  role IN ('engineer','admin','manager','security','noc_operator','field_tech')
                )
);
```

Таблица дата-центров:

```
CREATE TABLE datacenters (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(120) NOT NULL UNIQUE,
    location      VARCHAR(200) NOT NULL,
    tier_level    SMALLINT     NOT NULL CHECK (tier_level BETWEEN 1 AND 4),
    total_power_kw  NUMERIC(10,2) NOT NULL CHECK (total_power_kw > 0),
    contact_email   VARCHAR(150)  NOT NULL UNIQUE
);
```

Таблица стоек:

```
CREATE TABLE racks (
    id            SERIAL PRIMARY KEY,
    datacenter_id   INT         NOT NULL REFERENCES datacenters(id) ON DELETE
RESTRICT,
    label         VARCHAR(20)  NOT NULL,
    row_label     VARCHAR(10)  NOT NULL,
    position      SMALLINT     NOT NULL CHECK (position > 0),
    max_units     SMALLINT     NOT NULL DEFAULT 42 CHECK (max_units
BETWEEN 1 AND 52),
    max_power_w     INT         NOT NULL CHECK (max_power_w > 0),
    UNIQUE (datacenter_id, label)
);
```

Таблица климатической телеметрии стоек (IoT):

```
CREATE TABLE rack_climate_metrics (
   id           BIGSERIAL PRIMARY KEY,
   rack_id      INT         NOT NULL REFERENCES racks(id) ON DELETE CASCADE,
   recorded_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
   power_w      INT         NOT NULL CHECK (power_w >= 0),
   temperature_c  NUMERIC(4,1) NOT NULL CHECK (temperature_c BETWEEN 10.0
AND 45.0)
);
```

Таблица серверов:

```
CREATE TABLE servers (
   id              SERIAL PRIMARY KEY,
   rack_id         INT         NOT NULL REFERENCES racks(id) ON DELETE
RESTRICT,
   contract_id     INT         REFERENCES contracts(id) ON DELETE SET NULL,
   serial_number   VARCHAR(60)  NOT NULL UNIQUE,
   hostname        VARCHAR(100) NOT NULL UNIQUE,
   manufacturer    VARCHAR(80)  NOT NULL,
   model           VARCHAR(100) NOT NULL,
   status          VARCHAR(20)  NOT NULL DEFAULT 'active'
                   CHECK (status IN ('active','maintenance','decommissioned','reserved')),
   cpu_cores       SMALLINT     NOT NULL CHECK (cpu_cores > 0),
   ram_gb          SMALLINT     NOT NULL CHECK (ram_gb > 0),
   storage_tb      NUMERIC(8,2) NOT NULL CHECK (storage_tb >= 0),
   power_w         INT         NOT NULL CHECK (power_w > 0),
   rack_unit_position  SMALLINT     NOT NULL CHECK (rack_unit_position > 0),
   rack_units_height   SMALLINT     NOT NULL DEFAULT 1 CHECK
(rack_units_height BETWEEN 1 AND 10),
   installed_at    DATE        NOT NULL DEFAULT CURRENT_DATE,
   assigned_engineer   INT         REFERENCES employees(id) ON DELETE SET NULL,
   UNIQUE (rack_id, rack_unit_position)
);
```

Таблица IP-адресов (IPAM) с использованием специфичных типов PostgreSQL:

```
CREATE TABLE ip_addresses (
   id          SERIAL PRIMARY KEY,
   server_id   INT         REFERENCES servers(id) ON DELETE SET NULL,
   ip_address  INET        NOT NULL UNIQUE,
   subnet      CIDR        NOT NULL,
   ip_type     VARCHAR(20)  NOT NULL CHECK (ip_type IN
('management','production','internal','public')),
   is_vip      BOOLEAN     NOT NULL DEFAULT FALSE
);
```

Таблица инцидентов:

```
CREATE TABLE incidents (
   id           SERIAL PRIMARY KEY,
   server_id      INT         NOT NULL REFERENCES servers(id) ON DELETE
RESTRICT,
   reported_by    INT         NOT NULL REFERENCES employees(id) ON DELETE
RESTRICT,
   assigned_to    INT         REFERENCES employees(id) ON DELETE SET NULL,
   title         VARCHAR(200) NOT NULL,
   description    TEXT,
   severity       VARCHAR(10)  NOT NULL CHECK (severity IN
('low','medium','high','critical')),
   status         VARCHAR(20)  NOT NULL DEFAULT 'open'
                  CHECK (status IN ('open','in_progress','resolved','closed')),
   created_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
   resolved_at    TIMESTAMPTZ,
   CONSTRAINT resolved_after_created CHECK (resolved_at IS NULL OR resolved_at
>= created_at)
);
```

```
datacenter=# \dt+
                                    List of relations
 Schema |         Name          | Type  |  Owner  | Persistence | Access method |    Size    | Description
--------+-----------------------+-------+---------+-------------+---------------+------------+-------------
 public | contracts             | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | datacenters           | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | employees             | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | incident_services     | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | incident_updates      | table | dcadmin | permanent   | heap          | 16 kB      |
 public | incidents             | table | dcadmin | permanent   | heap          | 16 kB      |
 public | ip_addresses          | table | dcadmin | permanent   | heap          | 16 kB      |
 public | maintenance_logs      | table | dcadmin | permanent   | heap          | 16 kB      |
 public | rack_climate_metrics  | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | racks                 | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | server_components     | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | servers               | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | services              | table | dcadmin | permanent   | heap          | 8192 bytes |
 public | vendors               | table | dcadmin | permanent   | heap          | 8192 bytes |
(14 rows)
```

2.2.2 Добавление внешних ключей и ограничение целостности

В схеме реализован полный набор ограничений для обеспечения консистентности данных:

Первичные ключи (PRIMARY KEY) - определены для всех 14 таблиц и обеспечивают уникальность каждой записи, например, каждая таблица имеет автоинкрементный первичный ключ: **id SERIAL PRIMARY KEY**

Внешние ключи (FOREIGN KEY / REFERENCES) - 19 ссылочных ограничений. Три стратегии поведения при удалении:

ON DELETE RESTRICT - запрет удаления, если есть зависимые записи

Нельзя удалить дата-центр, пока в нём есть стойки
datacenter_id INT NOT NULL REFERENCES datacenters(id) ON DELETE RESTRICT

ON DELETE CASCADE - каскадное удаление дочерних записей

При удалении сервера автоматически удаляются его сервисы и компоненты
server_id INT NOT NULL REFERENCES servers(id) ON DELETE CASCADE

ON DELETE SET NULL - обнуление ссылки при удалении родителя

При удалении инженера серверы не удаляются, поле обнуляется
assigned_engineer INT REFERENCES employees(id) ON DELETE SET NULL

Ограничение уникальности (UNIQUE) - 12 ограничений на бизнес-ключи:

email VARCHAR(150) NOT NULL UNIQUE - уникальный e-mail сотрудника
phone VARCHAR(20) NOT NULL UNIQUE - уникальный телефон
hostname VARCHAR(100) NOT NULL UNIQUE - уникальное имя хоста
serial_number VARCHAR(60) NOT NULL UNIQUE - серийный номер сервера
ip_address INET NOT NULL UNIQUE - уникальный IP-адрес
UNIQUE (rack_id, rack_unit_position) - запрет двух серверов в одном юните стойки

Ограничения CHECK - 29 ограничений бизнес-логики:

Допустимый уровень надёжности дата-центра
tier_level SMALLINT NOT NULL CHECK (tier_level BETWEEN 1 AND 4)
Температура в гермозоне стойки - только разумный диапазон
temperature_c NUMERIC(4,1) NOT NULL CHECK (temperature_c BETWEEN 10.0 AND 45.0)

Допустимые статусы серверов - перечисление значений
status VARCHAR(20) NOT NULL CHECK (status IN ('active','maintenance','decommissioned','reserved'))

Допустимые типы компонентов
component_type VARCHAR(30) NOT NULL CHECK (component_type IN ('CPU','RAM','HDD','SSD','NIC','PSU'))

Порт сервиса - стандартный диапазон TCP/UDP
port INT CHECK (port BETWEEN 1 AND 65535)

Время разрешения инцидента не может быть раньше создания
CONSTRAINT resolved_after_created CHECK (resolved_at IS NULL OR resolved_at >= created_at)

Дата окончания контракта не может быть раньше начала
end_date DATE NOT NULL CHECK (end_date >= start_date)

Формат e-mail
email VARCHAR(150) NOT NULL UNIQUE CHECK (email LIKE '%@%.%')

Ограничение NOT NULL - 68 полей помечены как обязательные, что предотвращает появление неполных записей во всех критичных атрибутах.

```
 assigned_engineer  | integer                  |           |          |
Indexes:
    "servers_pkey" PRIMARY KEY, btree (id)
    "idx_servers_rack" btree (rack_id)
    "idx_servers_status" btree (status)
    "servers_hostname_key" UNIQUE CONSTRAINT, btree (hostname)
    "servers_rack_id_rack_unit_position_key" UNIQUE CONSTRAINT, btree (rack_id, rack_unit_position)
    "servers_serial_number_key" UNIQUE CONSTRAINT, btree (serial_number)
Check constraints:
    "servers_cpu_cores_check" CHECK (cpu_cores > 0)
    "servers_power_w_check" CHECK (power_w > 0)
    "servers_rack_unit_position_check" CHECK (rack_unit_position > 0)
    "servers_rack_units_height_check" CHECK (rack_units_height >= 1 AND rack_units_height <= 10)
    "servers_ram_gb_check" CHECK (ram_gb > 0)
    "servers_status_check" CHECK (status::text = ANY (ARRAY['active'::character varying, 'maintenance'::character varying, 'decommi
ssioned'::character varying, 'reserved'::character varying]::text[]))
    "servers_storage_tb_check" CHECK (storage_tb >= 0::numeric)
Foreign-key constraints:
    "servers_assigned_engineer_fkey" FOREIGN KEY (assigned_engineer) REFERENCES employees(id) ON DELETE SET NULL
    "servers_contract_id_fkey" FOREIGN KEY (contract_id) REFERENCES contracts(id) ON DELETE SET NULL
    "servers_rack_id_fkey" FOREIGN KEY (rack_id) REFERENCES racks(id) ON DELETE RESTRICT
Referenced by:
    TABLE "incidents" CONSTRAINT "incidents_server_id_fkey" FOREIGN KEY (server_id) REFERENCES servers(id) ON DELETE RESTRICT
    TABLE "ip_addresses" CONSTRAINT "ip_addresses_server_id_fkey" FOREIGN KEY (server_id) REFERENCES servers(id) ON DELETE SET NULL
    TABLE "maintenance_logs" CONSTRAINT "maintenance_logs_server_id_fkey" FOREIGN KEY (server_id) REFERENCES servers(id) ON DELETE
CASCADE
    TABLE "server_components" CONSTRAINT "server_components_server_id_fkey" FOREIGN KEY (server_id) REFERENCES servers(id) ON DELET
E CASCADE
    TABLE "services" CONSTRAINT "services_server_id_fkey" FOREIGN KEY (server_id) REFERENCES servers(id) ON DELETE CASCADE

datacenter=#
```

```
rack_climate_metrics          rack_climate_metrics_pkey      racks_datacenter_id_label_key  racks_pkey
rack_climate_metrics_id_seq   racks                          racks_id_seq
datacenter=# \d rack
rack_climate_metrics          rack_climate_metrics_pkey      racks_datacenter_id_label_key  racks_pkey
rack_climate_metrics_id_seq   racks                          racks_id_seq
datacenter=# \d racks
                                    Table "public.racks"
    Column     |         Type          | Collation | Nullable |              Default
---------------+-----------------------+-----------+----------+-----------------------------------
 id            | integer               |           | not null | nextval('racks_id_seq'::regclass)
 datacenter_id | integer               |           | not null |
 label         | character varying(20) |           | not null |
 row_label     | character varying(10) |           | not null |
 position      | smallint              |           | not null |
 max_units     | smallint              |           | not null | 42
 max_power_w   | integer               |           | not null |
Indexes:
    "racks_pkey" PRIMARY KEY, btree (id)
    "racks_datacenter_id_label_key" UNIQUE CONSTRAINT, btree (datacenter_id, label)
Check constraints:
    "racks_max_power_w_check" CHECK (max_power_w > 0)
    "racks_max_units_check" CHECK (max_units >= 1 AND max_units <= 52)
    "racks_position_check" CHECK ("position" > 0)
Foreign-key constraints:
    "racks_datacenter_id_fkey" FOREIGN KEY (datacenter_id) REFERENCES datacenters(id) ON DELETE RESTRICT
Referenced by:
    TABLE "rack_climate_metrics" CONSTRAINT "rack_climate_metrics_rack_id_fkey" FOREIGN KEY (rack_id) REFERENCES racks(id) ON DELET
E CASCADE
    TABLE "servers" CONSTRAINT "servers_rack_id_fkey" FOREIGN KEY (rack_id) REFERENCES racks(id) ON DELETE RESTRICT

datacenter=#
```

2.2.3 Создание индексов

Создано 10 индексов для ускорения аналитических запросов и операций поиска. Каждый индекс обоснован сценарием использования:

Частая фильтрация серверов по статусу (Q-02, Q-05, Q-10)
CREATE INDEX idx_servers_status ON servers(status);

JOIN между серверами и стойками (Q-01, Q-04, Q-05)
CREATE INDEX idx_servers_rack ON servers(rack_id);

Составной индекс для запросов по открытым инцидентам заданной критичности (Q-03, Q-07, Q-08)
CREATE INDEX idx_incidents_composite ON incidents(status, severity);

JOIN сервисов с серверами (Q-08, Q-16)
CREATE INDEX idx_services_server ON services(server_id);

Поиск по IP-адресу - тип INET требует отдельного индекса (Q-13)
CREATE INDEX idx_ip_addresses_search ON ip_addresses(ip_address);

Выборка компонентов конкретного сервера (Q-14, Q-17)
CREATE INDEX idx_components_server ON server_components(server_id);

Временной ряд IoT-метрик стойки - убывающий порядок (Q-15)
CREATE INDEX idx_climate_rack_time ON rack_climate_metrics(rack_id, recorded_at DESC);

Быстрый вывод worklog по инциденту (Q-20)
CREATE INDEX idx_incident_updates_inc ON incident_updates(incident_id);

JOIN журнала обслуживания с серверами
CREATE INDEX idx_maintenance_server ON maintenance_logs(server_id);

Выборка последних записей обслуживания по дате (Q-09)
CREATE INDEX idx_maintenance_performed ON maintenance_logs(performed_at DESC);

```
public | employees_phone_key                  | index | dcadmin | employees           | permanent | btree | 16 kB |
public | employees_pkey                       | index | dcadmin | employees           | permanent | btree | 16 kB |
public | idx_climate_rack_time                | index | dcadmin | rack_climate_metrics | permanent | btree | 16 kB |
public | idx_components_server                | index | dcadmin | server_components    | permanent | btree | 16 kB |
public | idx_incident_updates_inc             | index | dcadmin | incident_updates     | permanent | btree | 16 kB |
public | idx_incidents_composite              | index | dcadmin | incidents           | permanent | btree | 16 kB |
public | idx_ip_addresses_search              | index | dcadmin | ip_addresses        | permanent | btree | 16 kB |
public | idx_maintenance_performed            | index | dcadmin | maintenance_logs    | permanent | btree | 16 kB |
public | idx_maintenance_server               | index | dcadmin | maintenance_logs    | permanent | btree | 16 kB |
public | idx_servers_rack                     | index | dcadmin | servers             | permanent | btree | 16 kB |
public | idx_servers_status                   | index | dcadmin | servers             | permanent | btree | 16 kB |
public | idx_services_server                  | index | dcadmin | services            | permanent | btree | 16 kB |
public | incident_services_pkey               | index | dcadmin | incident_services   | permanent | btree | 16 kB |
public | incident_updates_pkey                | index | dcadmin | incident_updates    | permanent | btree | 16 kB |
public | incidents_pkey                       | index | dcadmin | incidents           | permanent | btree | 16 kB |
public | ip_addresses_ip_address_key          | index | dcadmin | ip_addresses        | permanent | btree | 16 kB |
public | ip_addresses_pkey                    | index | dcadmin | ip_addresses        | permanent | btree | 16 kB |
public | maintenance_logs_pkey                | index | dcadmin | maintenance_logs    | permanent | btree | 16 kB |
public | rack_climate_metrics_pkey            | index | dcadmin | rack_climate_metrics | permanent | btree | 16 kB |
public | racks_datacenter_id_label_key        | index | dcadmin | racks               | permanent | btree | 16 kB |
public | racks_pkey                           | index | dcadmin | racks               | permanent | btree | 16 kB |
public | server_components_pkey               | index | dcadmin | server_components    | permanent | btree | 16 kB |
public | server_components_serial_number_key  | index | dcadmin | server_components    | permanent | btree | 16 kB |
public | servers_hostname_key                 | index | dcadmin | servers             | permanent | btree | 16 kB |
public | servers_pkey                         | index | dcadmin | servers             | permanent | btree | 16 kB |
public | servers_rack_id_rack_unit_position_key | index | dcadmin | servers           | permanent | btree | 16 kB |
public | servers_serial_number_key            | index | dcadmin | servers             | permanent | btree | 16 kB |
public | services_pkey                        | index | dcadmin | services            | permanent | btree | 16 kB |
public | vendors_name_key                     | index | dcadmin | vendors             | permanent | btree | 16 kB |
public | vendors_pkey                         | index | dcadmin | vendors             | permanent | btree | 16 kB |
(36 rows)
```

2.3 Наполнение тестовыми данными (data.sql)

Скрипт data.sql наполняет базу репрезентативными данными, отражающими реалистичный сценарий импортозамещения: отечественные серверы YADRO и Булат используются совместно с legacy-оборудованием HPE и Dell. Данные подобраны так, чтобы все 20 запросов возвращали содержательные результаты.

2.3.2 Статистика наполнения

Таблица	Назначение	Записей
employees	Сотрудники	8
datacenters	Дата-центры	3
racks	Стойки	6
vendors	Вендоры	4
contracts	Контракты SLA	3
servers	Серверы	12
server_components	Компоненты	7
ip_addresses	IP-адреса (IPAM)	7
rack_climate_metrics	IoT-телеметрия	5
services	Сервисы	12
incidents	Инциденты	8
incident_updates	Рабочие заметки	2
incident_services	Связь инцидент↔сервис	10
maintenance_logs	Журнал обслуживания	8
ИТОГО		95

2.3.2 Примеры добавленных данных

Дата-центры:

```
INSERT INTO datacenters (name, location, tier_level, total_power_kw, contact_email)
VALUES
('DC-Alpha',  'Москва, ул. Складочная 1',   3, 2400.00, 'ops@dc-alpha.ru'),
('DC-Beta',   'Санкт-Петербург, Пулково',   4, 5000.00, 'ops@dc-beta.ru'),
('DC-Gamma',  'Новосибирск, Академгородок', 2,  800.00, 'ops@dc-gamma.ru');
```

Вендоры (отечественные производители):

```
INSERT INTO vendors (name, support_email, support_phone) VALUES
('YADRO (КНС Групп)', 'support@yadro.ru',  '+74957778899'),
('Элтекс (Eltex)',    'tech@eltex-co.ru',  '+73833350000'),
('Булат',             'support@bulat.ru',  '+74951234567'),
('Т-Ком',             'info@t-kom.ru',     '+74959998877');
```

Контракты SLA:

```
INSERT INTO contracts (vendor_id, contract_number, start_date, end_date, sla_hours)
VALUES
(1, 'CTR-YADRO-2024-01', '2024-01-01', '2027-01-01', 4), - 4ч реакция
(2, 'CTR-ELTEX-2024-11', '2024-05-15', '2026-05-15', 8), - 8ч реакция
(3, 'CTR-BULAT-2025-09', '2025-01-10', '2026-01-10', 12); - 12ч реакция
```

Серверы (примеры - импортозамещение и legacy):

```
INSERT INTO servers (rack_id, contract_id, serial_number, hostname,
   manufacturer, model, status, cpu_cores, ram_gb, storage_tb,
   power_w, rack_unit_position, rack_units_height, installed_at, assigned_engineer)
VALUES
(1, 1, 'SN-DELL-001', 'srv-web-01',  'YADRO', 'VEGMAN S220',   'active', 32, 256, 3.84,
750, 1,2,'2022-03-15',1),
(3, 1, 'SN-DELL-003', 'srv-api-01',  'YADRO', 'VEGMAN R120',   'active', 24, 192, 7.68,
600, 1,2,'2023-06-10',1),
(4, 3, 'SN-LEN-001',  'srv-vpn-01',  'Булат', 'БС-202',        'active', 16, 128, 3.84, 550,
1,2,'2022-07-18',5),
(5, 1, 'SN-DELL-005', 'srv-bkp-01',  'YADRO', 'TATLIN.ARCHIVE','active', 16,
128,48.00, 400, 2,2,'2022-12-12',1),
(3, NULL,'SN-DELL-004','srv-mon-01', 'Dell',  'PowerEdge
R350','maintenance',8,64,1.92,300, 3,1,'2020-09-05',8);
```

Компоненты с намеренно сбойным блоком питания (для демонстрации Q-14 и Q-17):

```
INSERT INTO server_components (server_id, component_type, model, serial_number,
status)
VALUES
(1, 'PSU', 'Delta Electronics 800W', 'PSU-DEL-011', 'healthy'),
(1, 'PSU', 'Delta Electronics 800W', 'PSU-DEL-012', 'failed'); - сбой на srv-web-01
```

IoT-телеметрия стоек (временные метки относительно NOW()):

```
INSERT INTO rack_climate_metrics (rack_id, recorded_at, power_w, temperature_c)
VALUES
(1, NOW() - INTERVAL '30 minutes', 1400, 21.2),
(1, NOW(),                         1500, 22.0),
(4, NOW() - INTERVAL '15 minutes', 3800, 24.5),
(4, NOW(),                         4100, 25.1),
(5, NOW(),                         600,  19.8);
```

VIP-адрес для MinIO Object Storage:

```
INSERT INTO ip_addresses (server_id, ip_address, subnet, ip_type, is_vip)
VALUES
(9, '85.249.10.45', '85.249.10.0/28', 'public', false),
(9, '85.249.10.50', '85.249.10.0/28', 'public', true);
```

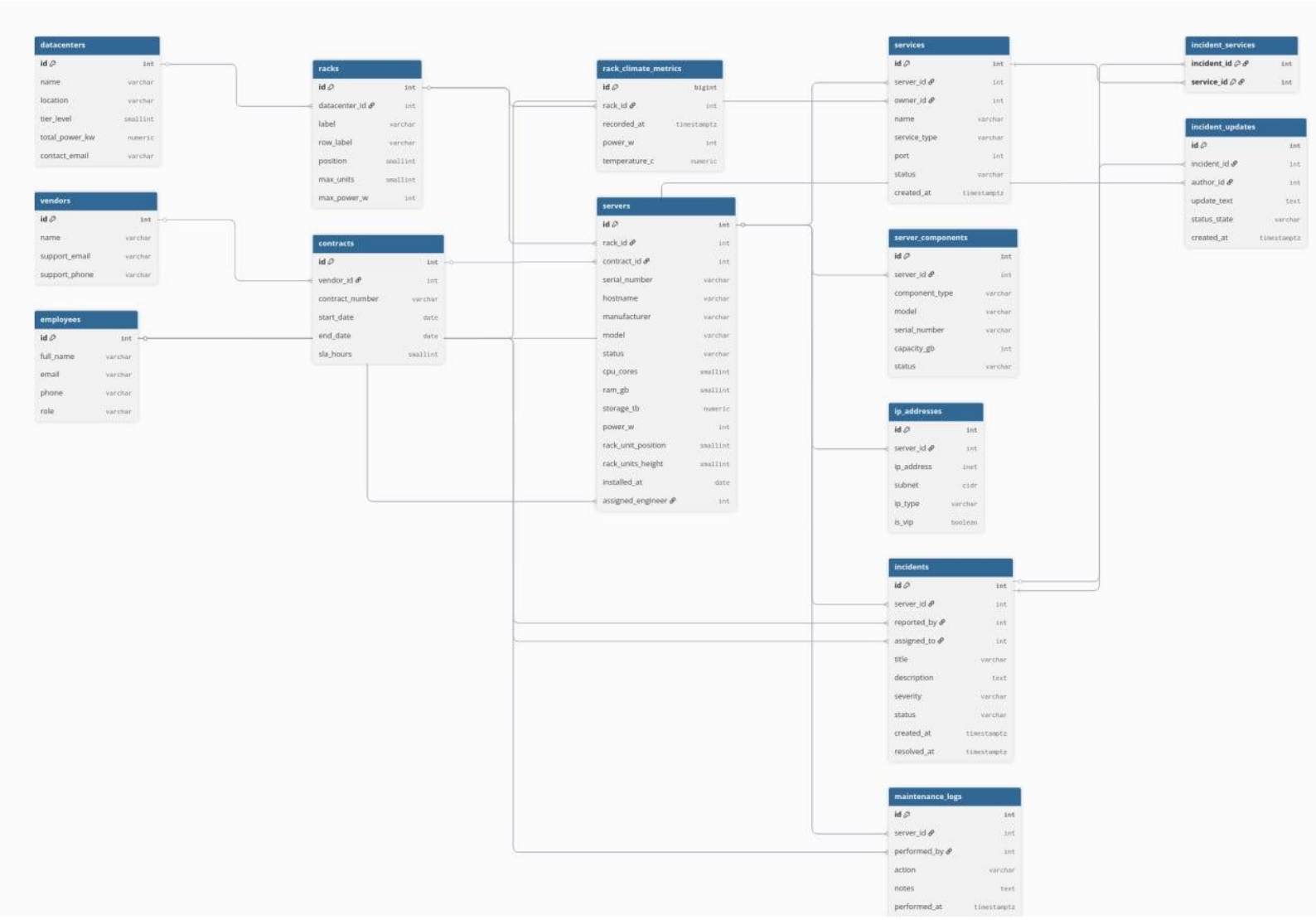
- VIP для MinIO

2.4 Визуализация структуры БД (diagram)

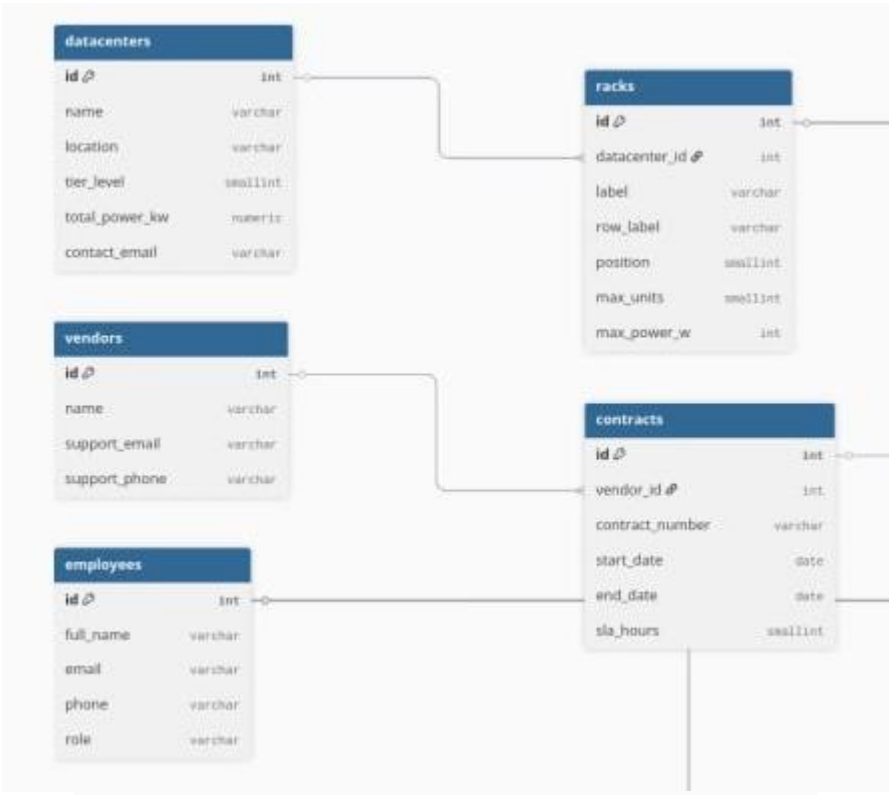
На основе созданной схемы построена ER-диаграмма логической структуры базы данных. Диаграмма отображает 14 таблиц и все связи между ними.

Описание связей:

Связь	Тип	Пояснение
Datacenters => racks	1:M	Один ДЦ содержит много стоек
racks => servers	1:M	Одна стойка вмещает много серверов
racks => rack_climate_metrics	1:M	Каждая стойка имеет историю метрик
vendors => contracts	1:M	Один вендор - несколько договоров
contracts => servers	1:M	Один договор покрывает несколько серверов
servers => server_components	1:M	Сервер содержит много компонентов (CASCADE)
servers => ip_addresses	1:M	Сервер может иметь несколько IP
servers => services	1:M	На одном сервере запущено несколько сервисов
servers => incidents	1:M	На одном сервере может быть много инцидентов
employees => servers	1:M	Инженер ответственен за несколько серверов
employees => incidents	1:M	Сотрудник регистрирует и берёт инциденты
incidents <=> services	M:N	Через таблицу incident_services
incidents => incident_updates	1:M	Worklog к инциденту (CASCADE)



Ниже диаграмма выше приведена более крупно, но отдельными частями


datacenters
id int
name varchar
location varchar
tier_level smallint
total_power_kw numeric
contact_email varchar
racks
id int
datacenter_id int
label varchar
row_label varchar
position smallint
max_units smallint
max_power_w int
vendors
id int
name varchar
support_email varchar
support_phone varchar
contracts
id int
vendor_id int
contract_number varchar
start_date date
end_date date
sla_hours smallint
employees
id int
full_name varchar
email varchar
phone varchar
role varchar


rack_climate_metrics
id bigint
rack_id int
recorded_at timestamptz
power_w int
temperature_c numeric
servers
id int
rack_id int
contract_id int
serial_number varchar
hostname varchar
manufacturer varchar
model varchar
status varchar
cpu_cores smallint
ram_gb smallint
storage_tb numeric
power_w int
rack_unit_position smallint
rack_units_height smallint
installed_at date
assigned_engineer int


services
id int
server_id int
owner_id int
name varchar
service_type varchar
port int
status varchar
created_at timestamptz
server_components
id int
server_id int
component_type varchar
model varchar
serial_number varchar
capacity_gb int
status varchar
ip_addresses
id int
server_id int
ip_address inet
subnet cidr
ip_type varchar
is_vip boolean
incidents
id int
server_id int
reported_by int
assigned_to int
title varchar
description text
severity varchar
status varchar
created_at timestamptz
resolved_at timestamptz
maintenance_logs
id int
server_id int
performed_by int
action varchar
notes text
performed_at timestamptz


incident_services
incident_id
int
service_id
int
incident_updates
id
int
incident_id
int
author_id
int
update_text
text
status_state
varchar
created_at
timestamptz

2.5 Аналитические запросы (queries.sql)

Разработано 20 аналитических SQL-запросов (Q-01..Q-20), покрывающих все требуемые конструкции языка.

2.5.1 Q-01 .. Q-10 - базовый и средний уровень

Q-01 - Полный каталог серверов (SELECT + JOIN, 3 таблицы)
Задача: Получить список всего оборудования с указанием дата-центра и стойки.

```
SELECT
    dc.name                      AS datacenter,
    r.label                      AS rack,
    s.hostname,
    s.manufacturer || ' ' || s.model   AS hardware,
    s.cpu_cores, s.ram_gb, s.storage_tb, s.status, s.installed_at
FROM servers s
JOIN racks r       ON r.id = s.rack_id
JOIN datacenters dc ON dc.id = r.datacenter_id
ORDER BY dc.name, r.label, s.rack_unit_position;
```

Q-02 - Серверы на обслуживании или в резерве (SELECT + WHERE)
Задача: Быстро найти неактивное оборудование и ответственного инженера.

```
SELECT s.hostname, s.status,
    s.manufacturer || ' ' || s.model AS hardware,
    e.full_name                  AS responsible_engineer
FROM servers s
LEFT JOIN employees e ON e.id = s.assigned_engineer
WHERE s.status IN ('maintenance', 'reserved')
ORDER BY s.status, s.hostname;
```

Q-03 - Открытые инциденты с назначенными сотрудниками (SELECT + WHERE + JOIN)
Задача: Оперативная сводка для дежурной смены NOC - что сейчас в работе.

```
SELECT i.id, i.severity, i.title,
    s.hostname                   AS affected_server,
    rep.full_name                AS reported_by,
    COALESCE(asgn.full_name, '-')      AS assigned_to,
    i.created_at
FROM incidents i
JOIN servers   s    ON s.id = i.server_id
JOIN employees rep  ON rep.id = i.reported_by
LEFT JOIN employees asgn ON asgn.id = i.assigned_to
WHERE i.status IN ('open', 'in_progress')
ORDER BY CASE i.severity
    WHEN 'critical' THEN 1 WHEN 'high' THEN 2
    WHEN 'medium'   THEN 3 ELSE 4 END, i.created_at;
```

Q-04 - Количество серверов по статусам в каждом дата-центре (GROUP BY + COUNT)

```
SELECT dc.name AS datacenter, s.status, COUNT(*) AS server_count
FROM servers s
JOIN racks r       ON r.id = s.rack_id
JOIN datacenters dc ON dc.id = r.datacenter_id
GROUP BY dc.name, s.status
ORDER BY dc.name, s.status;
```

Q-05 - Суммарное потребление мощности по стойкам (GROUP BY + SUM + HAVING)

Задача: Найти стойки, где нагрузка превышает 1500 Вт, для контроля энергоёмкости.

```
SELECT dc.name AS datacenter, r.label AS rack,
   r.max_power_w AS rack_limit_w,
   SUM(s.power_w) AS used_power_w,
   ROUND(SUM(s.power_w)::NUMERIC / r.max_power_w * 100, 1) AS load_pct
FROM servers s
JOIN racks r       ON r.id = s.rack_id
JOIN datacenters dc ON dc.id = r.datacenter_id
WHERE s.status = 'active'
GROUP BY dc.name, r.label, r.max_power_w
HAVING SUM(s.power_w) > 1500
ORDER BY load_pct DESC;
```

Q-06 - Среднее время разрешения инцидентов по severity (GROUP BY + AVG)

```
SELECT severity, COUNT(*) AS total_incidents,
   ROUND(AVG(EXTRACT(EPOCH FROM (resolved_at -
created_at))/3600)::NUMERIC, 2)
      AS avg_resolution_hours
FROM incidents
WHERE status IN ('resolved', 'closed') AND resolved_at IS NOT NULL
GROUP BY severity
ORDER BY CASE severity WHEN 'critical' THEN 1 WHEN 'high' THEN 2
   WHEN 'medium' THEN 3 ELSE 4 END;
```

Q-07 - Топ-3 сервера по числу инцидентов (CTE + ORDER BY LIMIT)

```
WITH incident_counts AS (
   SELECT s.id, s.hostname,
      s.manufacturer || ' ' || s.model AS hardware,
      COUNT(i.id)                    AS incident_count,
      SUM(CASE WHEN i.severity = 'critical' THEN 1 ELSE 0 END) AS critical_count
   FROM servers s
   LEFT JOIN incidents i ON i.server_id = s.id
   GROUP BY s.id, s.hostname, s.manufacturer, s.model
)
SELECT hostname, hardware, incident_count, critical_count
FROM incident_counts
ORDER BY incident_count DESC, critical_count DESC
```

```
LIMIT 3;
```

```
datacenter=# WITH incident_counts AS (
datacenter(#     SELECT s.id,
datacenter(#     s.hostname,
datacenter(#     s.manufacturer || ' ' || s.model AS hardware,
datacenter(#     COUNT(i.id) AS incident_count,
datacenter(#     SUM(CASE WHEN i.severity = 'critical' THEN 1 ELSE 0 END) AS critical_co
datacenter(#     FROM servers s
datacenter(#     LEFT JOIN incidents i ON i.server_id = s.id
datacenter(#     GROUP BY s.id, s.hostname, s.manufacturer, s.model
datacenter(# )
datacenter-# SELECT hostname,
datacenter-# hardware,
datacenter-# incident_count,
datacenter-# critical_count
datacenter-# FROM incident_counts
datacenter-# ORDER BY incident_count DESC, critical_count DESC
datacenter-# LIMIT 3;
  hostname   |     hardware      | incident_count | critical_count
-------------+-------------------+----------------+----------------
 srv-web-01  | YADRO VEGMAN S220 |              2 |              1
 srv-stor-01 | Булат БС-404      |              1 |              1
 srv-api-01  | YADRO VEGMAN R120 |              1 |              0
(3 rows)
```

Q-08 - Сервисы, затронутые авариями critical/high (подзапрос IN + JOIN)

```
SELECT DISTINCT svc.name AS service, svc.service_type, svc.status, srv.hostname AS host
FROM services svc
JOIN servers srv ON srv.id = svc.server_id
WHERE svc.id IN (
   SELECT is2.service_id FROM incident_services is2
   JOIN incidents i ON i.id = is2.incident_id
   WHERE i.severity IN ('critical', 'high')
)
ORDER BY svc.service_type, svc.name;
```

```
datacenter=# SELECT DISTINCT
datacenter-# svc.name AS service,
datacenter-# svc.service_type,
datacenter-# svc.status,
datacenter-# srv.hostname AS host
datacenter-# FROM services svc
datacenter-# JOIN servers srv ON srv.id = svc.server_id
datacenter-# WHERE svc.id IN (
datacenter(#     SELECT is2.service_id
datacenter(#     FROM incident_services is2
datacenter(#     JOIN incidents i ON i.id = is2.incident_id
datacenter(#     WHERE i.severity IN ('critical', 'high')
datacenter(# )
datacenter-# ORDER BY svc.service_type, svc.name;
       service        | service_type | status  |    host
----------------------+--------------+---------+-------------
 PostgreSQL Primary   | database     | running | srv-db-01
 PostgreSQL Replica   | database     | running | srv-db-02
 MinIO Object Storage | storage      | running | srv-stor-01
 OpenVPN              | vpn          | running | srv-vpn-01
(4 rows)

datacenter=#
```

Q-09 - Последние 5 записей журнала обслуживания (JOIN + ORDER BY LIMIT)

```
SELECT ml.performed_at, s.hostname, e.full_name AS technician,
   ml.action, ml.notes
FROM maintenance_logs ml
JOIN servers   s ON s.id = ml.server_id
JOIN employees e ON e.id = ml.performed_by
ORDER BY ml.performed_at DESC
LIMIT 5;
```

Q-10 - Сводка нагрузки RAM по дата-центрам (GROUP BY + SUM/AVG)

```
SELECT dc.name AS datacenter, dc.tier_level,
   COUNT(s.id)                AS active_servers,
   SUM(s.ram_gb)              AS total_ram_gb,
   ROUND(AVG(s.ram_gb)::NUMERIC, 1) AS avg_ram_gb,
   SUM(s.cpu_cores)           AS total_cpu_cores
FROM servers s
JOIN racks r       ON r.id = s.rack_id
JOIN datacenters dc ON dc.id = r.datacenter_id
WHERE s.status = 'active'
GROUP BY dc.name, dc.tier_level
ORDER BY total_ram_gb DESC;
```

2.5.2 Q-11 .. Q-20 - расширенный аналитический стек

Q-11 - Ранжирование серверов по объёму памяти внутри ДЦ (оконная функция DENSE_RANK)

Задача: Выявить наиболее мощные ноды на каждой площадке для оптимального распределения нагрузки.

```
SELECT dc.name AS dc_name, s.hostname, s.ram_gb,
   DENSE_RANK() OVER (PARTITION BY dc.id ORDER BY s.ram_gb DESC) AS
ram_rank
FROM servers s
JOIN racks r ON r.id = s.rack_id
JOIN datacenters dc ON dc.id = r.datacenter_id;
```

Q-12 - Контроль нарушений SLA по договорам (CTE + DATE-операции)

Задача: Найти инциденты, время решения которых превысило гарантированное договором SLA.

```
WITH incident_duration AS (
   SELECT id, server_id,
      EXTRACT(EPOCH FROM (resolved_at - created_at))/3600 AS actual_hours
   FROM incidents WHERE status = 'resolved' AND resolved_at IS NOT NULL
)
SELECT s.hostname, v.name AS vendor_name, c.contract_number,
   ROUND(idr.actual_hours::NUMERIC, 1) AS hours_taken,
   c.sla_hours AS promised_sla
FROM incident_duration idr
JOIN servers s ON s.id = idr.server_id
JOIN contracts c ON c.id = s.contract_id
```

```
JOIN vendors v ON v.id = c.vendor_id
WHERE idr.actual_hours > c.sla_hours;
```

Q-13 - Аудит сетевой безопасности: публичные и VIP-адреса (INET/CIDR)

Задача: Вывести все адреса продакшн-периметра для аудита безопасности.

```
SELECT s.hostname, ip.ip_address, ip.subnet, ip.ip_type, ip.is_vip
FROM ip_addresses ip
JOIN servers s ON s.id = ip.server_id
WHERE ip.ip_type = 'public' OR ip.is_vip = TRUE;
```

Q-14 - Дефектная ведомость: серверы со сбойным оборудованием (JOIN + WHERE)

```
SELECT s.hostname, sc.component_type, sc.model, sc.serial_number, sc.status
FROM server_components sc
JOIN servers s ON s.id = sc.server_id
WHERE sc.status = 'failed';
```

Q-15 - Скользящее среднее температуры стоек (оконная функция AVG ROWS BETWEEN)

Задача: Сгладить температурные пики для детекции проблем с кондиционированием гермозоны.

```
SELECT r.label AS rack_label, cl.recorded_at, cl.temperature_c,
   ROUND(AVG(cl.temperature_c) OVER (
      PARTITION BY cl.rack_id
      ORDER BY cl.recorded_at
      ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
   ), 2) AS sliding_avg_temperature
FROM rack_climate_metrics cl
JOIN racks r ON r.id = cl.rack_id;
```

```
datacenter=# SELECT
datacenter-#     r.label AS rack_label,
datacenter-#     cl.recorded_at,
datacenter-#     cl.temperature_c,
datacenter-#     ROUND(AVG(cl.temperature_c) OVER (
datacenter(#         PARTITION BY cl.rack_id
datacenter(#         ORDER BY cl.recorded_at
datacenter(#         ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
datacenter(#     ), 2) AS sliding_avg_temperature
datacenter-# FROM rack_climate_metrics cl
datacenter-# JOIN racks r ON r.id = cl.rack_id;
 rack_label |         recorded_at          | temperature_c | sliding_avg_temperature
------------+------------------------------+---------------+-------------------------
 A-01       | 2026-05-20 17:20:44.34639+00 |          21.2 |                   21.20
 A-01       | 2026-05-20 17:50:44.34639+00 |          22.0 |                   21.60
 R-01       | 2026-05-20 17:35:44.34639+00 |          24.5 |                   24.50
 R-01       | 2026-05-20 17:50:44.34639+00 |          25.1 |                   24.80
 R-02       | 2026-05-20 17:50:44.34639+00 |          19.8 |                   19.80
(5 rows)
```

Q-16 - Матрица состояния сервисов по типам (условная агрегация GROUP BY + FILTER)

Задача: Верхнеуровневая бизнес-метрика здоровья всего сервисного ландшафта.

```
SELECT service_type,
   COUNT(*) FILTER (WHERE status = 'running')  AS running_count,
   COUNT(*) FILTER (WHERE status = 'degraded') AS degraded_count,
   COUNT(*) FILTER (WHERE status = 'stopped')  AS stopped_count
FROM services
GROUP BY service_type;
```

Q-17 - Серверы без резервного питания (GROUP BY + HAVING)

Задача: Найти серверы с менее чем двумя исправными блоками питания - риск падения при отключении питания.

```
SELECT s.hostname, COUNT(sc.id) AS active_psu_count
FROM servers s
LEFT JOIN server_components sc
   ON s.id = sc.server_id AND sc.component_type = 'PSU' AND sc.status = 'healthy'
GROUP BY s.id, s.hostname
HAVING COUNT(sc.id) < 2;
```

Q-18 - KPI инженеров: топ по закрытым тикетам (JOIN + GROUP BY + LIMIT)

```
SELECT e.full_name, e.role, COUNT(i.id) AS resolved_incidents_count
FROM employees e
JOIN incidents i ON e.id = i.assigned_to
WHERE i.status IN ('resolved', 'closed')
GROUP BY e.id, e.full_name, e.role
ORDER BY resolved_incidents_count DESC
LIMIT 1;
```

Q-19 - Контракты с истекающим сроком (CURRENT_DATE)

Задача: Найти договоры, истекающие в ближайшие 365 дней, для инициирования пролонгации.

```
SELECT v.name AS vendor, c.contract_number, c.end_date,
   (c.end_date - CURRENT_DATE) AS days_until_expiration
FROM contracts c
JOIN vendors v ON v.id = c.vendor_id
WHERE c.end_date - CURRENT_DATE < 365;
```

Q-20 - Таймлайн Post-Mortem по инциденту (иерархический JOIN)

Задача: Полная хронология работы по инциденту #5 для разбора ошибок.

```
SELECT up.created_at, e.full_name AS technical_author,
   up.update_text, up.status_state
FROM incident_updates up
JOIN employees e ON e.id = up.author_id
WHERE up.incident_id = 5
ORDER BY up.created_at ASC;
```

2.6 Запуск через Docker Compose

Для обеспечения воспроизводимости окружения и упрощения проверки разработан файл docker-compose.yml, разворачивающий PostgreSQL 16 с автоматической инициализацией схемы и данных.

2.6.1 Файл конфигурации

```
version: "3.9"

services:
  postgres:
    image: postgres:16-alpine
    container_name: datacenter_db
    restart: unless-stopped
    environment:
      POSTGRES_USER: dcadmin
      POSTGRES_PASSWORD: dcpass123
      POSTGRES_DB: datacenter
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./schema.sql:/docker-entrypoint-initdb.d/01_schema.sql:ro
      - ./data.sql:/docker-entrypoint-initdb.d/02_data.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dcadmin -d datacenter"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 15s

volumes:
  pgdata:
    driver: local
```

2.6.2 Порядок запуска

Шаг	Команда	Назначение
1	**docker compose up -d**	Запуск контейнера в фоновом режиме
2	**docker compose ps**	Проверка статуса контейнера (ожидать healthy)
3	**psql -h localhost -U dcadmin -d datacenter**	Подключение через psql
4	**docker compose exec postgres psql -U dcadmin -d datacenter**	Подключение через Docker exec
5	**docker compose down**	Остановка (данные сохраняются в томе)
6	**docker compose down -v**	Полная очистка: остановка + удаление

2.6.3 Автоматическая инициализация

При первом запуске контейнера PostgreSQL автоматически выполняет SQL-файлы из каталога /docker-entrypoint-initdb.d/ в алфавитном порядке:

- schema.sql - создание структуры таблиц, ограничений и индексов
- data.sql - наполнение тестовыми данными (95 тестовых записей, распределенных по 14 таблицам)

Благодаря политике restart: unless-stopped контейнер автоматически перезапускается после перезагрузки системы. Healthcheck проверяет готовность PostgreSQL к приёму подключений каждые 10 секунд, что позволяет корректно выстраивать зависимости в более сложных Compose-конфигурациях.

Подключение к базе данных в DBeaver:

```
Host:     localhost
Port:     5432
Database: datacenter
User:     dcadmin
Password: dcpass123
```

2.7 Результат выполнения

В результате выполнения работы создана полностью функциональная база данных для управления IT-инфраструктурой дата-центра, включающая:

- 14 таблиц, связанных 19 внешними ключами - значительно превышает требования всех уровней
- 95 тестовых записей по всем таблицам
- 10 индексов для оптимизации производительности аналитических запросов
- 29 CHECK-ограничений, 12 UNIQUE, 68 NOT NULL - полное обеспечение целостности данных
- 20 аналитических SQL-запросов (Q-01..Q-20)
- Покрытие всех обязательных конструкций: SELECT, WHERE, JOIN, GROUP BY, HAVING, ORDER BY LIMIT
- Продвинутые конструкции PostgreSQL: оконные функции DENSE_RANK и AVG OVER ROWS BETWEEN, работа с типами INET/CIDR
- Docker Compose для воспроизводимого развёртывания с автоматической инициализацией

3. Выводы

В данной расчётно-графической работе была спроектирована и реализована реляционная база данных для управления IT-инфраструктурой дата-центра (DCIM). Разработанная схема включает 14 таблиц, описывающих полный жизненный цикл серверного оборудования: от физического размещения в дата-центрах и стойках до управления инцидентами, контроля SLA и IoT-телеметрии.

Особое внимание уделено актуальной теме импортозамещения: в тестовых данных используются отечественные производители YADRO, Элтекс и Булат наряду с legacy-оборудованием. Это позволяет демонстрировать реалистичные сценарии управления смешанной инфраструктурой в условиях перехода на российские решения.

Схема данных обеспечивает строгую целостность через 19 внешних ключей с тремя стратегиями поведения (CASCADE, RESTRICT, SET NULL), 29 CHECK-ограничений, реализующих бизнес-логику, и 12 ограничений уникальности. Десять индексов оптимизируют наиболее нагруженные запросы.

Разработанные 20 аналитических запросов демонстрируют широкий спектр возможностей PostgreSQL: от базовых операций JOIN и агрегации до оконных функций DENSE_RANK и скользящего среднего, а также специфичных типов данных INET/CIDR для управления сетевой адресацией. Запрос Q-12 реализует практически значимую задачу контроля нарушений SLA с использованием CTE и функций работы с временными интервалами.

База данных развёрнута в контейнере Docker на основе образа PostgreSQL 16-alpine. Конфигурация docker-compose.yml обеспечивает автоматическую инициализацию схемы и тестовых данных при первом запуске, healthcheck для контроля готовности сервиса и постоянное хранение данных в именованном томе.